

U2T02: SimCSE, train your own sentence embedding model

BERT encodes tokens well and sentences badly. If you pool its vectors and compare sentences by cosine similarity, almost everything looks similar to almost everything, so the embeddings are useless for retrieval or semantic comparison. SimCSE (Gao, Yao and Chen, 2021) fixes this with a contrastive objective that reshapes the space, and it does so with remarkably little machinery: in its unsupervised version, the only data augmentation is dropout.

In this assignment, your main task is to replicate the SimCSE training method, both the unsupervised and the supervised version, starting from bert-base-uncased. At the end, each team owns a sentence embedding model it trained and published on the Hugging Face Hub. You will not reproduce the paper's numbers with the data we give you, and that is deliberate: understanding why your numbers are lower is a central part of the work.

Part 1: Understand the objective

Identify how the contrastive loss works (Eq. 1 of the paper; Eq. 5 for the supervised case):

- In Eq. 1, what is in the numerator, what does the denominator sum over, and with your chosen batch size, how many negatives does one optimization step push the positive away from?
- The temperature τ appears in Eq. 1. What does it control, and what did the paper find when it ablated it?
- How does this objective differ from the one Sentence-BERT was trained with (2019), and why does that difference matter for an STS-B evaluation?
- How does the training objective relate to the alignment and the uniformity of the embeddings it produces?

Part 2: The data

You train on a subset of SNLI (<https://huggingface.co/datasets/stanfordnlp/snli>) provided: snli_train_100k.jsonl, 100k records of premise, hypothesis and label (0 = entailment, 1 = neutral, 2 = contradiction), taken from the official train split by dropping records without annotator consensus and sampling 100k records. Every team works from the same file. If you want more data, get it yourself and say so in your report.

From this one file you build the two training sets:

- Unsupervised: the 165,529 unique sentences.
- Supervised: the 33,351 (premise, entailment hypothesis) pairs. For about 28% of them there is also a contradiction hypothesis for the same premise; those are the paper's hard negatives.

For evaluation, download STS-B (<https://huggingface.co/datasets/sentence-transformers/stsb>). Use dev (1,500 pairs) while you develop and to pick your best checkpoint; use test (1,379 pairs) exactly once, for the final number in your benchmark table.

Part 3: Train both modes

Implement the two versions of SimCSE as the paper describes them: unsupervised (§3) and supervised (§4, with hard negatives), training on the SNLI subset from Part 2, not on the paper's data.

Every training decision is yours: pooling, learning rate, batch size, temperature, epochs, dropout rate, seeds. List what you used in your report.

During training, evaluate on the STS-B dev set and keep the best checkpoint. For every run, save a small JSON file with the configuration, seed, hardware and results, and keep a simple log of all your runs. If something breaks, a loss that will not move or collapses to zero, figure out why; what happened and how you solved it belongs in the report.

Part 4: Evaluate

Evaluate every model as follows: embed both sentences of each pair, normalize, take the cosine similarity, and correlate the similarities with the human scores using Spearman. No regressor on top; the paper's Appendix B explains why these choices matter.

Beyond the Spearman number, each model needs the analysis that explains it: alignment and uniformity (Wang & Isola), the distribution of cosine similarities grouped by human rating, and a few nearest-neighbor retrievals, including at least one failure case you discuss.

Before your first training run, check that your evaluation code is correct: raw bert-base-uncased and the SBERT-2019 checkpoint give fixed, reproducible numbers, so score one of them first. If your result is far from the reference values in the Notes, the bug is in your evaluation code; fix it now, not after hours of training

Part 5: Ablations

Run one ablation per mode:

- Unsupervised: use the same dropout mask for both views.
- Supervised: hard negatives on vs. off.

In each case exactly one thing changes between the two runs. Report both configurations, the dev and test deltas, whether the difference is larger than what run-to-run noise would explain, and why the change has the effect you measured.

Part 6: Benchmark and the gap

Build one table with dev Spearman, test Spearman, alignment and uniformity for:

- your two SimCSE models,
- raw bert-base-uncased,
- SBERT-2019 (bert-base-nli-mean-tokens), evaluated, not retrained,
- the paper's reported STS-B numbers, comparing dev with dev and test with test.

Then write the most important paragraph of the report: account for the distance between your numbers and the paper's. The data we gave you differs from the paper's in several ways; identify them. How much of the gap do they explain, and how much comes from your setup? Put numbers on it.

Part 7: Publish

Export your best model as a sentence-transformers model and push it to the Hub. Then verify it: reload it from the Hub, rerun your STS-B test evaluation, and check that you get the same number as in your table. Write a model card with the training data, your recipe, the metrics and the limitations.

Part 8: Optional

More data, other datasets, other ablations: your call. Declare whatever you do.

Deliverables

- Your code: training, evaluation, export. Pinned dependencies.
- Your published models on the Hub and the log of your runs.
- Your report: the Part 1 answers, your configuration table, the benchmark table with its analysis (alignment/uniformity plot, similarity distributions, retrievals), the two ablations, and the gap discussion.

Minimum complete delivery: both models trained, your own evaluation with a dev and a test number, the benchmark table, a short report covering the required sections, and at least one model on the Hub verified by reloading it. Anything beyond this improves your grade.

Notes

Reference values (STS-B, Spearman $\times 100$): raw bert-base-uncased with mean pooling, 59.31 dev / 47.29 test; SBERT-2019 (bert-base-nli-mean-tokens), 80.77 / 76.98.

SimCSE: Gao, Yao & Chen, SimCSE: Simple Contrastive Learning of Sentence Embeddings, EMNLP 2021 (<https://arxiv.org/abs/2104.08821>).

Sentence-BERT: Reimers & Gurevych, 2019 (<https://arxiv.org/abs/1908.10084>).

Wang & Isola, Alignment and Uniformity on the Hypersphere, ICML 2020 (<https://arxiv.org/abs/2005.10242>).

DO NOT FORGET TO INCLUDE YOUR TEAM MEMBER NAMES ON THE REPORT.